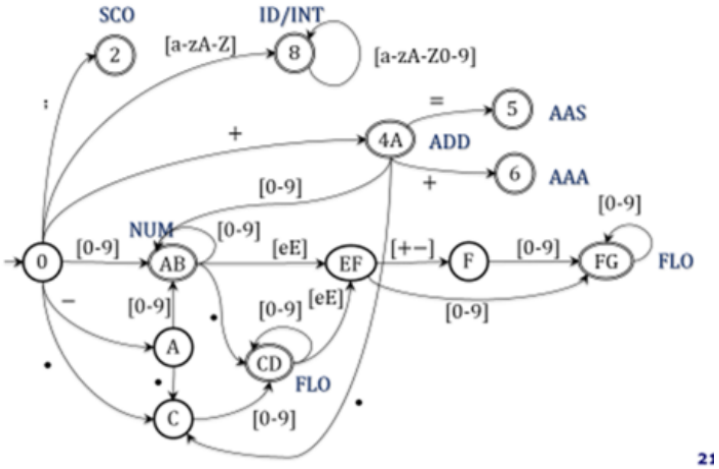
实验二：实现某种类 C 语言的词法分析器

根据第一次实验写一个复杂的 DFA，能够识别某种类 C 语言的词法单位。



已知某类 C 语言的文法，写一个扫描器，能够识别该语言的所有词法单位。

P → Ď Š

Ď → ε | Ď D ;

D → T d | T d[i] | T d (Ă) {Ď Š}

T → int | float | void

Ă → ε | Ă A ;

A → T d | T d[] | T d(T)

Š → S | Š ; S

S → d = E | d[E] = E | if (B) S | if (B) S else S | while (B) S | return E | {Š} | d(Ř) |

E → i | f | d | d[E] | E + E | E * E | (E) | d(Ř)

B →E r E | E

Ř → ε | Ř R ,

R → E | d[] | d()

变元都是语法单元。程序 P；声明表 Ď；声明 D；类型 T；形参表 Ă；形参 A；Ř 实参表；实参 R；语句表 Š；语句 S；表达式 E；布尔表达式 B。

终结符为词法单元。d（语言 ID）；整数 i（语言 NUM）；实数 f（语言 FLO）；算术运算符 o（语言{+, -, *, /}）；关系运算符 r（语言{<, >, ==, <=, >=, !=}；以此类推。

词法记号列表:

d ~ (ID, name)

ID = [a-z][a-z0-9]*

i ~ (INT, value)

NUM = [+-]?[0-9]+

f ~ (FLO, value)

FLO = [+-]?[0-9]*. [0-9]+ | [+-]?[0-9]+. [0-9]*

+ ~ (ADD, -)

ADD = {+}

* ~ (MUL, -)

MUL = {*}

r ~ (ROP, name)

ROP = {<, <=, ==}

\= ~ (ASG, -)

ASG = {\=}

\(~ (LPA, -)

LPA = {\(}

\) ~ (RPA, -)

RPA = {\)}

[~ (LBK, -)

LBK = {[}

] ~ (RBK, -)

RBK = {]}

\{ ~ (LBR, -)

LBR = {\{}

\} ~ (RBR, -)

RBR = {\}}

\, ~ (CMA, -)

CMA = {\,}

\; ~ (SCO, -)

SCO = {\;}

int ~ (INT, -)

float ~ (FLOAT, -)

if ~ (IF, -)

else ~ (ELSE, -)

while ~ (WHILE, -)

return ~ (RETURN, -)

input ~ (INPUT, -)

print ~ (PRINT, -)

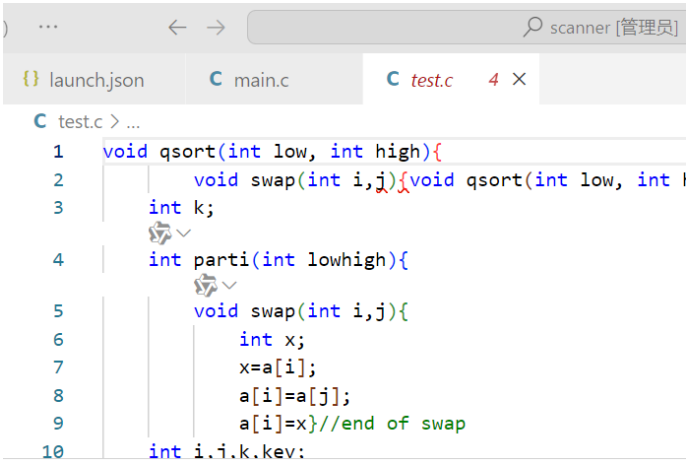
void ~ (VOID,-)

实现步骤和要求

利用 DFA 实现上述词法分析器 scanner（），能够识别该类 C 语言的 token 元素（必做）

1、例如写一个测试文件 test.c 词法分析器输出如上述右边

2、将终端输出到一个文件（目的是将词法分析的结果作为后续分析的输入文件）




```
(VOID, void)
(ID, qsort)
(LPA, ()
(INT, int)
(ID, low)
(CMA, ,)
(INT, int)
(ID, high)
(RPA, ))
(LBR, {)
(VOID, void)
(ID, swap)
(LPA, ()
(INT, int)
(ID, i)
(CMA, ,)
(ID, j)
(RPA, ))
(LBR, {)
```